PYTHON PROGRAMMING / Written Assignments

The objective of these weekly written assignments is to guide you to ***systematically familiarise yourself with the course materials*** and help you to ***learn fundamental concepts more deeply***.

Please write your short answers to separate Word or PDF documents (each topic to a separate document) called **PP_T***n_yourSurname*`.docx` or `.pdf`
where *n* is the topic number.
* Please submit the documents to Moodle.

Topic T1

1.1 What if there is a syntax error in your Python code, does your program crash due to the syntax error? Give arguments!

1.2 Python does not use braces `{}` to indicate a block of code to be executed when a condition is true. How do you tell Python which statements are included in a block of code that is to be executed when a loop condition is true?

1.3 There are eight symbols in the Python code below.

```
if x < 2026:
    y = 42
```

What identifiers, keywords, literals, operators, and separators (delimiters) you can find in the code?
=> NB! Be especially careful with the *operators*!

1.4 How do you create a variable in Python?

1.5 What is the Python naming convention for variables, functions, methods, scripts, and modules?

1.6 How do you write a comment in Python?

1.7 What is the maximum recommended code line length in Python?

1.8 What if your code line would be longer than recommended. How can you split the original code line over more than one physical line?

1.9 By default, the print() function moves the cursor to next line each time when we call it. How can you print several times on the same line?

1.10 What are f-strings in Python?

1.11 What are the Python data type names of the following literals:

```
a)    2026
b)    202.6
c)    "2026"
d)    True
```

1.12 Is the program below correct? Give arguments. (Not printing anything is not a problem)

```
s = s + 2
```

NB! You can run the code in the interactive Python environment as follows:
1. Start the interactive Python interpreter by typing **py** to the Windows search box.
2. Copy/paste the code to the interpreter window.

1.13 Is the program below correct? Give arguments.

```
s = "1"
s = s + 2
```

1.14 Is the program below correct? Give arguments.

```
s = "1"
s = 2
s = s + 2
```

1.15 Is the code below correct? Give arguments! Maybe you should try to run the code.

```
x = input("Enter an integer: ")
x = x * 5
if x > 0:
    print(x)
```

1.16 How can we convert a string to an int in Python?

1.17 How can we concatenate a string and an int in Python?

Topic T2

2.1 What does the code below print? NB! The objective is to ***understand why Python gives certain results***. Therefore, give arguments!

```
print(8 % 2)          # a)
print(8 % 1.5)        # b)
print(4.0 * 2.0 / 4)  # c)
print(4 * 2 / 4)      # d)
print(2 / 4 * 4)      # e)
print(2 // 4 * 4)     # f)
print(int(1.99))      # g)
print(3 * 1.1)        # h)
print(5 < 6)          # i)
print(2 + 5 < 6)      # j)
print(2 + (5 < 6))    # k)
```

NB! You can run the above code in the interactive Python environment as follows:
1. Start the interactive Python interpreter by typing **py** to the Windows search box.
2. Copy/paste the code to the interpreter window.

2.2 How can we perform accurate arithmetic computations in Python (without installing any extra libraries)?

2.3 When should we use the elif keyword?

2.4 How can we write a *very short test on a single line of code* to determine if the value of a variable called age is one of the following five values: 1, 7, 18, 22, and 45?

Optional, more advanced questions (grade level 5)

2.5 In Python, there is no assignment operator. Instead, assignment is a statement in Python. Please explain in your own words how does assignment in Python differ from assignment in JavaScript (or Java).

2.6 In Python, comparison operators have the same precedence. What are the values of the expressions below? Explain in detail why these two expressions evaluate to different values.

```
1 < 2 == True
True == 1 < 2
```

Topic T3

3.1 What is wrong in the code below?

```
row = 0
while row < 5:
    print(row)
```

3.2 What is the standard way to write a *clean while loop*?

3.3 How does the for loop in Python differ from the counter-controlled for loop in JavaScript (or Java)?

3.4 What does the range() function do?

3.4 By default, the program crashes when the Python interpreter executes the following code line:

```
age = int("ten years")
```

Explain how we can prevent the program to crash when Python executes the code line above. Assume that we do not modify the code line.

Topic T4

4.1 What should a minimal *function header* contain?

4.2 In Python, we use consistently the term ***parameter*** and the term ***argument***. How do these two differ from each other?

4.3 How do ***positional arguments*** differ from ***keyword arguments***?

4.4 How can we specify a default value for a parameter?

4.5 Why would it be a good idea to use ***type hints*** in a function header?

4.6 What can we do with the ***pass*** statement?

4.7 What is a '*library module'*? What kind of benefit we can get by creating and using 'library modules'?

4.8 When should we use the ***import*** statement?

4.9 What does the code below do?

```
if __name__ == "__main__":
    main()
```

4.10 What can we do with the **doctest** module?

Optional questions

4.11 Do all Python functions return something? Give arguments! (Please see the course material first)

Topic T5

5.1 Is there anything ***wrong*** in the code below?

```
numbers = [3, 1, 4, 5]
numbers[1] = 5
numbers[4] = 2
```

NB! You can run the code in the interactive Python environment as follows:

1. Start the interactive Python interpreter by typing **py** to the Windows search box.
2. Copy/paste the code to the interpreter window.

5.3 Is there anything ***wrong*** in the code below?

```
numbers = [3, 1, 4, 5]
numbers[-2] = 1
```

5.3 Suppose the following list:

```
numbers = [3, 1, 4, 5]
```

How can we print the items in the list as below without writing any loop?

```
3, 1, 4, 5
```

5.4 How can we get the **length** (number of elements) of a list?

5.5 How can we easily find the **maximum value** in the list below without writing any loop or sorting the list?

```
numbers = [3, 1, 4, 5]
```

5.6 How can we easily find the **sum of the values** in the list below without writing any loop or using the addition operator?

```
numbers = [3, 1, 4, 5]
```

5.7 How can we easily **determine if the value of 5 exists in the list** below without writing any loop?

```
numbers = [3, 1, 4, 5]
```

5.8 How can we create an **empty list**?

5.9 How can we **remove duplicates** from the list below without writing any loop?

```
names = ["cat", "horse", "cat", "bird", "cat", "horse"]
```

5.10 What does the code below do?

```
a = [2, 0, 2, 2]
b = [1]
print(a + b)
```

5.11 What does the code below do? Please give a detailed explanation.

```
a = [4, 2, 9]
b = [4, 100]
print(a > b)
```

5.12 What does the code below do? Please give a detailed explanation.

```
a = ["4", "2", "9"]
b = ["4", "100"]
print(a > b)
```

5.13 What is the main difference between sorting a list using the **sorted**() function and sorting a list using the **sort**() method.

5.14 How can we create (by writing only one line of code) a list of 500 items where each item is zero?

Topic T6

6.1 What kind of issue we have in the code below?

```
s = "abc"
s.upper()
print(s)
```

6.2 Is the following true or false: In Python, a single character is treated as a string of length one.

6.3 How can we test if a certain substring exists in a string?

6.4 How can we find the character in a string that has the highest character code (without sorting the characters)?

6.5 What do the following string handling methods do:
a) **strip**()
b) **capitalize**()

6.6 How do the string handling methods **find** and **rfind** differ from each other?

6.7 How can we check if a string contains only letters?

6.8 What is string ***slicing***?

6.9 Can we apply ***slicing*** on lists, too?

6.10 What does the code below print?

```
s = "STOP"
a = 1
b = 3
print(s[a:b])
print(s[1:3])
print(s[:])
print(s[1:])
print(s[:1])
print(s[::2])
print(s[::-3])
print(s[::-1])
```

6.11 How do the methods **split** and **splitlines** differ from each other?

6.12 What does the below print?

```
s = "A B C"
print(s.split())
```

6.13 What does the **join** method do?

6.14 What does the below print? Please explain in detail what happens in the code.

```
x = "1,2,3"
c = ","
print(x)
y = x.split(c)
z = c.join(y)
print(z)
```

Topic T7

7.1 What does the code below print?

```
a = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

print(a[1])
print(a[2][2])
```

7.2 What does the code below print?

```
teams = [["Sue", "John"], ["Ann", "Susan"], ["Bo", "Dan", "Jo"]]
for team in teams:
    print(*team, sep=", ")
```

7.3 Please visualise the code from **question 7.1** in the Python Tutor tool: https://pythontutor.com/visualize.html

a) Take a screen snip of the Python Tutor page that shows the code on the left and the **final visualisation** on the right. Finally, paste the screen snip to your answer.

b) How many lists you can see in the visualisation?

7.4 Please visualise the code from **question 7.2** in the Python Tutor tool: https://pythontutor.com/visualize.html

a) Take a screen snip of the Python Tutor page that shows the code on the left and the **final visualisation** on the right. Finally, paste the screen snip to your answer.

b) How many lists you can see in the visualisation?

7.5 Please visualise the following code in the Python Tutor tool:

```
teams = [["Sue", "John"], ["Ann", "Susan"], ["Bo", "Dan", "Jo"]]

teams.insert(0, ["Joe", "Li"])
teams.append(teams[1])
teams.pop(1)

for team in teams:
    print(*team, sep=", ")
```

NB! Please proceed slowly and see what happens on each step when you are examining the code and its visualisation. Pay special attention to the way how the content of the *team* variable changes during the program execution.

a) Take a screen snip of the Python Tutor page that shows the code on the left and the **final visualization** on the right. Finally, paste the screen snip to your answer.

b) How many lists will be created when the program is executed?

7.6 For practical purposes *the Python Tutor tool takes a shortcut to keep its visualisations tidy*. That is, it does not visualise everything exactly as it happens in the reality. Please explain what kind of shortcut the tool takes when it visualises Python programs.

7.7 How many string objects Python saves in the memory when it creates the list below.

```
a = ["A", "A", "A", "B"]
```

You can verify your answer by first creating the list and then running the code below. The id() function returns the 'location' of the item in the memory.

```
for item in a:
    print(id(item))
```

Topic T8

8.1 a) What does the code below print? Explain what has happened.

```
big_country = ["India", 1626037127260]
small_country = ["Tuvalu", 12119]

small_country = big_country
small_country[1] = 12222

print(f"The population of the small country is {small_country[1]}")
print(f"The population of {big_country[0]} is {big_country[1]}")
```

b) Please visualise the code in the Python Tutor tool: https://pythontutor.com/visualize.html

⇒ Take a screen snip of the Python Tutor page that shows the code on the left and the **final visualisation** on the right. Finally, paste the screen snip to your answer.

8.2 a) What is wrong in the program below? Explain what has happened.

```
nordic_countries = ["Finland", "Sweden", "Norway", "Denmark", "Iceland"]

european_countries = nordic_countries
european_countries.append("Spain")

print("European countries: ", end="")
print(*european_countries)

print("Nordic countries: ", end="")
print(*nordic_countries)
```

b) Please visualise the code in the Python Tutor tool: https://pythontutor.com/visualize.html

⇒ Take a screen snip of the Python Tutor page that shows the code on the left and the **final visualisation** on the right. Finally, paste the screen snip to your answer.

8.3 a) What does the code below print? What makes this program to print correct results?

```
nordic_countries = ["Finland", "Sweden", "Norway", "Denmark", "Iceland"]

european_countries = nordic_countries[:]
european_countries.append("Spain")

print("European countries: ", end="")
print(*european_countries)

print("Nordic countries: ", end="")
print(*nordic_countries)
```

b) Please visualise the code in the Python Tutor tool: https://pythontutor.com/visualize.html

⇒ Take a screen snip of the Python Tutor page that shows the code on the left and the **final visualisation** on the right. Finally, paste the screen snip to your answer.

8.4 a) What is wrong in the program below? Explain what has happened.

```
team_1 = ["Hawks", []]

team_2 = team_1.copy()
team_2[0] = "Tigers"

team_1[1].append("Ann")
team_1[1].append("Jill")

team_2[1].append("Susan")

print(*team_1)
print(*team_2)
```

b) Please visualise the above code in the Python Tutor tool: https://pythontutor.com/visualize.html

⇒ Take a screen snip of the Python Tutor page that shows the code on the left and the **final visualisation** on the right. Finally, paste the screen snip to your answer.

8.5 a) What does the code below print? What makes this program to print correct results?

```
from copy import deepcopy

team_1 = ["Hawks", []]

team_2 = deepcopy(team_1)
team_2[0] = "Tigers"

team_1[1].append("Ann")
team_1[1].append("Jill")

team_2[1].append("Susan")

print(*team_1)
print(*team_2)
```

b) Please visualise the above code in the Python Tutor tool: https://pythontutor.com/visualize.html

⇒ Take a screen snip of the Python Tutor page that shows the code on the left and the **final visualisation** on the right. Finally, paste the screen snip to your answer.

8.6 What is the main difference between the *shallow copy* and the *deep copy*?

Topic T9

9.1 What are the five built-in *collection types* in Python 3.10?

9.2 Can we iterate instances of all built-in collection types and strings using the for statement?

9.3 Do all built-in collection types and strings support numeric indexing and slicing?

9.4 What are the main differences between lists and tuples? Please consider Python's ***syntax and semantics*** only.

9.5 For what purposes Python itself uses tuples?

9.6 What are *sequence unpacking* and *sequence packing*?

9.7 In this question, we have the following two variables:

```
a = 1
b = 2
```

Change the code below so that you replace the first three statements with a single statement.

```
temp = a
a = b
b = temp
print(f"{a} {b}")
```

9.8 Explain what is *fundamentally* wrong in the code below.

```
student_list = ["John", "Harriet"]
student_tuple = ("Cathy", "Ann")

student_list.append("Peter")
student_tuple.append("Peter")
print(student_list)
print(student_tuple)
```

9.9 Explain why the code below works fine although tuples are immutable.

```
student_tuple = ("John", ["Finnish", "Swedish"])
print(student_tuple)
student_tuple[1].append("English")
print(student_tuple)
```

9.10 How do *named tuples* differ from ordinary tuples?

Topic T10

10.1 What is the main feature, which differentiates dictionaries from all other types of collections?

10.2 Please see the *overview of collection types* in the course material.

a) Do dictionaries allow duplicate items?

b) Do dictionaries keep items in any specific order in Python?

c) Can we iterate dictionaries using the for statement?

d) Can we apply slicing on dictionaries?

10.3 a) Can a value in a dictionary be of any data type?

b) Can a dictionary key be of any data type?

10.4 Suppose the dictionary below.

```
d = {"a": 10, "b": 20, "c": 30}
```

a) How can we print the value of the item whose key is "b"?

b) How can we add a new item to the dictionary? The key should be "e" and the value should be 40.

c) How can we replace the value of the item whose key is "b"? The new value should be 15.

d) How can we remove an item whose key is "a"?

e) How can we avoid a KeyError when we want to print the value of an item whose key is "xyz"?

10.5 How can we

a) get all items (key-value pairs) from a dictionary?

b) get all keys (keys only, no values) from a dictionary?

c) get all values (values only, no keys) from a dictionary?

10.6 a) What does the code below print?

```
countries = {}

countries["Finland"] = {"name": "Suomi", "area": 338_472}
countries["Sweden"] = {"name": "Sverige", "area": 440_295}

print(countries["Finland"]["name"])
print(countries["Finland"]["area"])

print(countries)
print(*countries)
```

b) How many dictionaries does the above program create?

c) Please visualise the above code in the Python Tutor tool: https://pythontutor.com/visualize.html

⇒ Take a screen snip of the Python Tutor page that shows the code on the left and the **final visualisation** on the right. Finally, paste the screen snip to your answer.

10.7 a) What does the code below print?

```
countries = {}

countries["Finland"] = {"name": "Suomi", "area": 338_472,
                        "capitol": {"name": "Helsinki", "population": 660_821}}
countries["Sweden"] = {"name": "Sverige", "area": 440_295,
                       "capitol": {"name": "Stockholm", "population": 978_770}}

print(countries["Finland"]["capitol"]["name"], end=", ")
print(countries["Finland"]["name"])

print(countries["Sweden"]["capitol"]["name"], end=", ")
print(countries["Sweden"]["name"])
```

b) How many dictionaries does the above program create?

c) Please visualise the above code in the Python Tutor tool: https://pythontutor.com/visualize.html

⇒ Take a screen snip of the Python Tutor page that shows the code on the left and the **final visualisation** on the right. Finally, paste the screen snip to your answer.

Topic T11

11.1 Which of the following are ***text files***: MS Word files, CSS files, PDF files, Python source files, Windows command files (.cmd files), MS Excel files, CSV files.

11.2 Using the ***Path*** class

a) What does `read_text(encoding="UTF-8")` do?

b) What does `write_text("Python", encoding="UTF-8")` do?

11.3 How can we check if the file that we are trying to read is found?

11.4 How can we use the Path class to get the ***current script's folder path***?

11.5 Why is the **splitlines**() method very useful when we are accessing a text file?

11.6 Why is the **split**() method very useful when we are accessing a CSV file?

11.7 When are **CSV files** very useful?